Pattern Matcher

AlgoExpert · Hard · Strings

The problem

- Pattern of 'x'/'y'; find substrings x and y so substituting yields `str`.
- Returns [x, y] (empty strings if unused) or [] if impossible.
- Normalise so pattern starts with 'x', then try every length of x.

Intuition

- Fix the length of x; the length of y follows from the counts, so test each candidate.

Brute force vs optimal

- Try all (len x, len y) pairs → $O(n^2 \cdot m)$
- Fix len(x); len(y) is determined → $O(n \cdot m)$
- Swap x/y if pattern starts with y

Approach

- 1. Normalise so the pattern starts with x
- 2. Count x and y
- 3. For each length of x, derive length of y
- 4. Build and compare

Worked example

```
pattern xxyxxy: 4 × x, 2 × y

string length 30
```

Count x and y

Worked example

```
try len(x)=1: y = (30 - 4)/2 = 13 → x='g' … ✗

try len(x)=2: y = 11 → x='go', y='powerranger' ✓
```

Try each length of x

Worked example

```
go go powerranger go go powerranger ✓

→ ['go', 'powerranger']
```

Match: x = go, y = powerranger

Java solution

```java
public class PatternMatcher {
    public static String[] patternMatcher(String pattern, String str) {
        if (pattern.length() > str.length()) return new String[]{};
        boolean swapped = pattern.charAt(0) != 'x';
        String p = swapped ? pattern.replace('x', '#').replace('y', 'x').replace('#', 'y') : pattern;
        int cx = 0, cy = 0, firstY = -1;
        for (int i = 0; i < p.length(); i++) {
            if (p.charAt(i) == 'x') cx++; else { cy++; if (firstY == -1) firstY = i; }
        }
        for (int lx = 1; lx * cx <= str.length(); lx++) {
            int rest = str.length() - lx * cx;
            if (cy == 0 ? rest != 0 : rest % cy != 0) continue;
            int ly = cy == 0 ? 0 : rest / cy;
            String x = str.substring(0, lx);
            String y = cy == 0 ? "" : str.substring(firstY * lx, firstY * lx + ly);
            StringBuilder sb = new StringBuilder();
            for (char c : p.toCharArray()) sb.append(c == 'x' ? x : y);
            if (sb.toString().equals(str)) return swapped ? new String[]{y, x} : new String[]{x, y};
        }
        return new String[]{};
    }
}
```

Complexity

- Time: O(n^2 + m)
- Space: O(n + m)

Edge cases & pitfalls

- Pattern starting with y: swap and swap back
- Pattern with only x
- Return in [x, y] order

Interview tips & follow-ups

- Explain why len(y) is determined

Recap

- Fix the length of x; the length of y follows from the counts, so test each candidate.
- Complexity: O(n^2 + m)